

Module 01: Getting Started

You already know how to program. This module gets Java running on your machine, shows you the smallest program that does something, and explains what actually happens when you run it. Half an hour, and you will never again be confused about the difference between the JDK, the JRE, and the JVM.

What You'll Learn

- How to get JDK 27 running and confirm you have the right one
- What the JVM, the JRE, and the JDK each are, and why people mix them up
- How to write and run a Java program without a class declaration or a build tool
- What the compiler does with your file, shown rather than asserted
- How to read a compiler error and find the exact spot it is complaining about

Coming From Another Language

In Python or JavaScript you write a file and run it. One step. Java traditionally made you do two: compile the source into bytecode, then run the bytecode. That extra step is where the "Java is heavyweight" reputation started.

Both of those things have changed, and this module is where you meet the change.

	Python / JavaScript	Java, traditionally	Java 27
Run a script	<code>python app.py</code>	<code>javac App.java</code> then <code>java App</code>	<code>java App.java</code>
Minimum program	<code>print("hi")</code>	<code>class</code> + <code>static main</code> + <code>String[] args</code>	<code>void main()</code> + <code>IO.println</code>
Errors found	at runtime	at compile time	at compile time

That last row is the one that will actually change how you work. Python tells you about a typo in a rarely-taken branch when a user hits it in production. Java tells you before the program starts. That is the trade you are making: more ceremony up front, fewer surprises later.

The Lesson

The three-letter words

People use JVM, JRE, and JDK interchangeably and then confuse each other. They are three different things stacked on top of each other:

- **JVM**, Java Virtual Machine. The thing that executes bytecode. It is why compiled Java runs unchanged on macOS, Linux, and Windows. Your code does not target a processor, it targets this.
- **JRE**, Java Runtime Environment. The JVM plus the standard library. Enough to *run* Java, not enough to compile it. You rarely install one separately now.
- **JDK**, Java Development Kit. The JRE plus the tools: `javac` the compiler, `javap` the disassembler, `jar`, `javadoc`. This is what you install.

You installed a JDK. Confirm it:

```
java -version
```

You want `java version "27"`. Anything else and the examples here may fail for reasons that have nothing to do with the lesson.

The smallest program

Open `examples/HelloWorld.java`. The entire file:

```
void main() {  
    IO.println("Hello, World!");  
}
```

If you have seen Java before, notice what is not there. No `public class`, no `static`, no `String[] args`, no import for the printing. That boilerplate was Java's most-mocked feature for twenty-five years, and JEP 512 removed the need to type it in JDK 25. It is stable, not preview.

`IO` is available without an import because it lives in `java.lang`, which every Java file imports automatically.

The boilerplate did not disappear

This is the part worth slowing down for, because it prevents a misconception that would bite you in Module 05.

Java did not grow a second, simpler language for beginners. The compiler writes the ceremony for you. Prove it:

```
javac -d /tmp/java27 modules/01-getting-started/examples/WhatTheCompilerWrites.java  
javap -cp /tmp/java27 WhatTheCompilerWrites
```

What comes back:

```
final class WhatTheCompilerWrites {  
    WhatTheCompilerWrites();  
    void main();  
}
```

There is your class. The compiler named it after the file, marked it `final`, gave it a no-argument constructor, and left `main` as an **instance** method, which is why the feature is called "instance main methods". The runtime builds one object of that class and calls `main` on it.

So you are writing ordinary Java the entire time, just typing less of it. Everything here transfers to the classic form in Module 05, because it *is* the classic form.

What running actually does

When you run `java HelloWorld.java`, three things happen in order:

1. `javac` compiles your source into **bytecode**, an instruction set the JVM understands. Since JDK 11 this can happen in memory, which is why you see no `.class` file appear.
2. The JVM loads that bytecode and verifies it.
3. The JVM executes it, compiling hot paths to native machine code as it goes.

Step 1 is where your typos are caught. This is the whole bargain of a compiled, statically typed language: the compiler reads every line before any of it runs.

You can still do it the long way, and sometimes you will want to:

```
javac Greet.java    # produces Greet.class
java Greet          # note: no .java, no .class, just the class name
```

Reading what the compiler tells you

Your first real skill in a new language is not writing code. It is reading the error and knowing where to look.

`examples/ReadTheError.java` is broken on purpose. Run it:

```
ReadTheError.java:29: error: ';' expected
    IO.println("I am missing something")
                        ^
1 error
```

Read it backwards from the caret:

- `^` points at the exact column
- `:29` is the line
- `';' expected` is what the compiler wanted to find there

Java is not guessing at your intent. It parsed your code up to that caret, and at that position the only legal token was a semicolon. Fix where the caret points, not the whole line.

Run It

```
# The minimal program
java modules/01-getting-started/examples/HelloWorld.java

# See what the compiler generated for you
java modules/01-getting-started/examples/WhatTheCompilerWrites.java

# Interactive, so run this one yourself
java modules/01-getting-started/examples/AskingForInput.java

# Broken on purpose. Read the error.
java modules/01-getting-started/examples/ReadTheError.java

# Check every example in this module still behaves
./scripts/verify-examples.sh modules/01-getting-started
```

Common Mistakes

Running `java Greet` when you only have `Greet.java`.

```
Error: Could not find or load main class Greet
Caused by: java.lang.ClassNotFoundException: Greet
```

`java Greet` looks for compiled bytecode named `Greet`. `java Greet.java` runs the source. The `.java` is the difference, and the error message does not spell that out for you.

Typing `io.println` instead of `IO.println`.

```
error: cannot find symbol
  io.println("hi");
  ^
symbol:   variable io
location: class E3
```

Java is case sensitive, and `IO` is a class name. "Cannot find symbol" is the error you will see most often in your first month. It nearly always means a typo, a missing import, or a name that does not exist yet.

Expecting `+` to add when the left side is text.

```
"Next year is " + year + 1    // "Next year is 20261"
"Next year is " + (year + 1)  // "Next year is 2027"
```

`+` evaluates left to right. Once one side is a `String`, every `+` after it means "append", not "add". Parenthesise the arithmetic.

Assuming `System.out.println` is obsolete. It is not. `IO.println` is shorthand that exists in compact source files. Every real codebase you will read uses `System.out.println` or a logging

framework, and both still work here.

Key Takeaways

- **The JDK is what you install.** It contains the JVM that runs bytecode and the tools that produce it. `java -version` must say 27 for this curriculum.
- **`java File.java` runs a source file directly.** No build tool, no separate compile step, no `.class` file left behind.
- **Compact source files are real Java.** The compiler writes the class declaration, the constructor, and the `final` modifier for you. `javap` shows you exactly what it wrote.
- **`main` here is an instance method**, so `this` exists. The classic `public static void main` does not have that, and Module 05 explains why.
- **Compile errors are located, not vague.** The caret points at the column, the number gives the line, and the message states what the compiler expected.
- **Nothing was replaced.** `System.out.println`, classes, and `static main` all still work. You are learning less typing, not a different language.

Homework

[homework/README.md](#)

Build a small interactive program from what you have just seen, then break it on purpose and read what the compiler says. Reference solution lives in [solutions/01-getting-started/](#), but try it first. Reading the answer teaches you much less than getting the error and fixing it.